Q21 Discussion Questions

1. MoE avoids the interface problem by training router and experts jointly in one loss — but that forces all experts to share one architecture, data stream, and objective. Is there a principled middle ground between "one giant differentiable program" and "frozen components behind a learned router," or is every intermediate point just a worse version of one extreme?
2. SayCan multiplies an LLM's estimate that a skill is *useful* by a learned value function's estimate that it is *possible* [**?**]. When the two estimates disagree, which should win — and can you name a concrete task where each ordering is the right one?
3. Compounding error argues for end-to-end training [**?**]; debuggability argues for pipelines. Autonomous-driving stacks have cycled between these positions for a decade. What evidence would actually settle the argument for a given domain — and is it the same evidence for a 100-robot fleet and a one-off research demo?
4. The interface trichotomy (embeddings / typed APIs / natural language) frames language as the lowest-bandwidth, most universal option. As LLMs get cheaper and faster, does the bandwidth penalty of language shrink, or does it stay fundamental because text is inherently lossy for continuous control signals?
5. Options formalize hierarchy with a clean semi-MDP interface [**?**], yet modern LLM agents compose skills with no such formalism — just prompts and tool schemas. What exactly did the options framework buy that prompt-based orchestration lost, and is any of it recoverable without giving up the flexibility?